



G L O B A L R A I N

Practices for Secure Software Report

Table of Contents

DOCUMENT REVISION HISTORY	3
CLIENT.....	3
INSTRUCTIONS.....	3
DEVELOPER	4
1. ALGORITHM CIPHER	4
2. CERTIFICATE GENERATION	4
3. DEPLOY CIPHER.....	5
4. SECURE COMMUNICATIONS	6
5. SECONDARY TESTING.....	7
6. FUNCTIONAL TESTING	8
7. SUMMARY	9
8. INDUSTRY STANDARD BEST PRACTICES.....	9

Document Revision History

Version	Date	Author	Comments
1.0	JUNE 16, 2026	ERNEST KOFI LOGAH	

Client



1. Algorithm Cipher

SHA-256 was selected as the cryptographic hash algorithm for Artemis Financial because it provides strong data integrity protection and is widely accepted as an industry standard. SHA-256 generates a fixed 256-bit hash value from any input data. If even one character changes, the resulting hash changes completely, making it useful for verifying data integrity. Unlike encryption algorithms, SHA-256 is a one-way hashing function and cannot be reversed to reveal the original data. SHA-256 was chosen instead of older algorithms such as MD5 and SHA-1 because those algorithms are vulnerable to collision attacks. The implementation uses Java's MessageDigest class to generate a checksum that can be used to verify that data has not been altered during transmission. In addition to hashing, modern security systems often use both symmetric and asymmetric cryptography. Symmetric encryption uses the same key for encryption and decryption and is typically faster for protecting large amounts of data. Asymmetric encryption uses a public and private key pair and is commonly used for secure communications and certificate-based authentication. The SSL certificate generated for this project uses a 2048-bit RSA key pair, which is an asymmetric cryptographic algorithm. Random values generated during cryptographic operations help strengthen security by making keys and encrypted communications more difficult to predict or compromise.

2. Certificate Generation

A self-signed SSL certificate was generated using Java Keytool. The certificate was stored in a PKCS12 keystore named keystore.p12 and exported as server.cer. The certificate contains a 2048-bit RSA key and supports secure HTTPS communication between clients and the application. The certificate allows encrypted communication during testing and demonstrates the use of secure communications within the application.

```
resources — -zsh — 80x24
twoeleven@Ernest's-MacBook-Air resources % keytool -printcert -file server.cer
Owner: CN=Ernest Kofi Logah, OU=CS305, O=SNHU, L=Atlanta, ST=GA, C=US
Issuer: CN=Ernest Kofi Logah, OU=CS305, O=SNHU, L=Atlanta, ST=GA, C=US
Serial number: 818a214edea2c329
Valid from: Tue Jun 16 17:49:05 EDT 2026 until: Wed Jun 16 17:49:05 EDT 2027
Certificate fingerprints:
    SHA1: BB:FB:95:5A:98:57:B2:E8:8C:8C:57:0A:8C:EA:82:3A:DD:1C:0E:F1
    SHA256: 6F:05:54:79:46:EA:EE:00:09:73:26:F9:AB:BC:AB:E6:09:64:22:9E:6F:
4E:7A:BA:71:7F:AC:0E:74:B1:85:72
Signature algorithm name: SHA384withRSA
Subject Public Key Algorithm: 2048-bit RSA key
Version: 3

Extensions:

#1: ObjectId: 2.5.29.14 Criticality=false
SubjectKeyIdentifier [
KeyIdentifier [
0000: E8 F9 29 AC 9C 28 3A 42    2E 63 9C A9 62 F7 E5 E0    ..)..(:B.c..b...
0010: 80 A7 3A 16                                ...:
]
]
```

Figure 1. Verification of the exported self-signed certificate (server.cer) generated using Java Keytool.

3. Deploy Cipher

The application was refactored to deploy SHA-256 checksum functionality through a new endpoint. A unique data string containing project information is processed using SHA-256 and returned to the browser along with the generated checksum. This checksum can be compared before and after transmission to verify that the data has not been modified. The implementation uses `MessageDigest.getInstance("SHA-256")` and converts the resulting hash into a hexadecimal string.

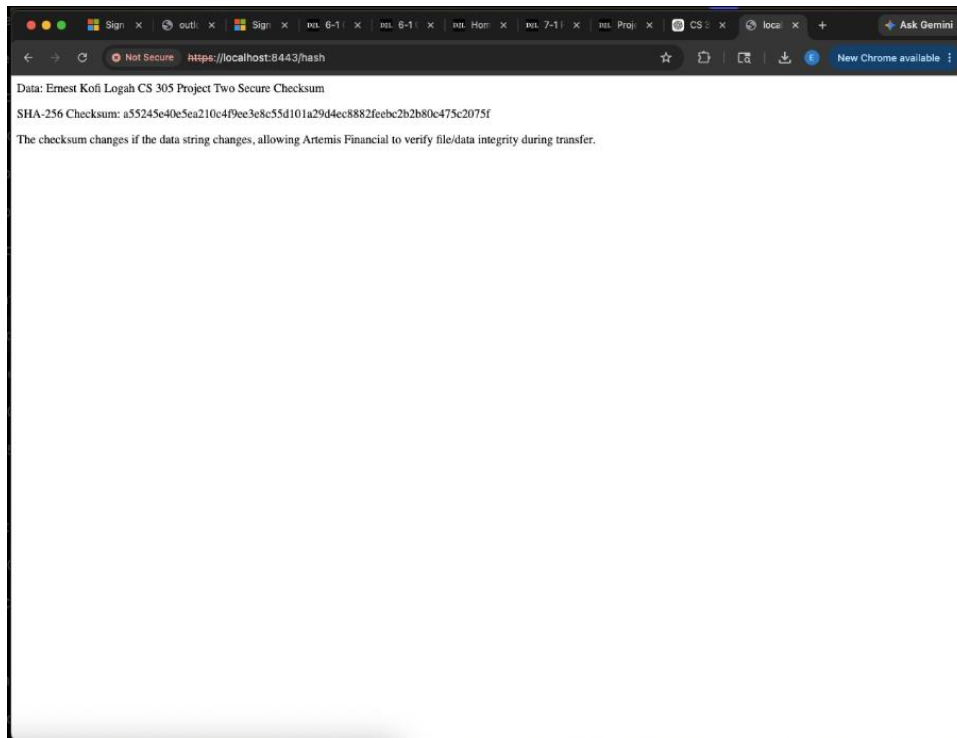


Figure 2. SHA-256 checksum verification demonstrating successful deployment of the cryptographic hash function.

4. Secure Communications

The application was converted from HTTP to HTTPS by modifying application.properties and configuring SSL support. The application now uses port 8443 and loads the SSL certificate from the PKCS12 keystore. HTTPS protects customer information by encrypting data while it is transmitted between the client and server. This reduces the risk of interception, eavesdropping, and man-in-the-middle attacks.

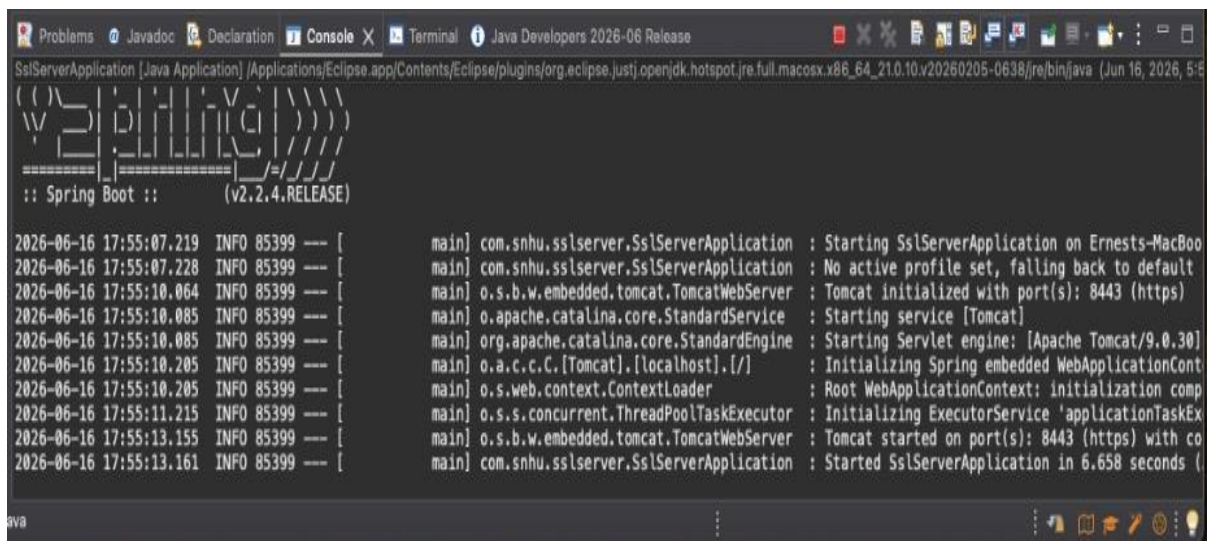
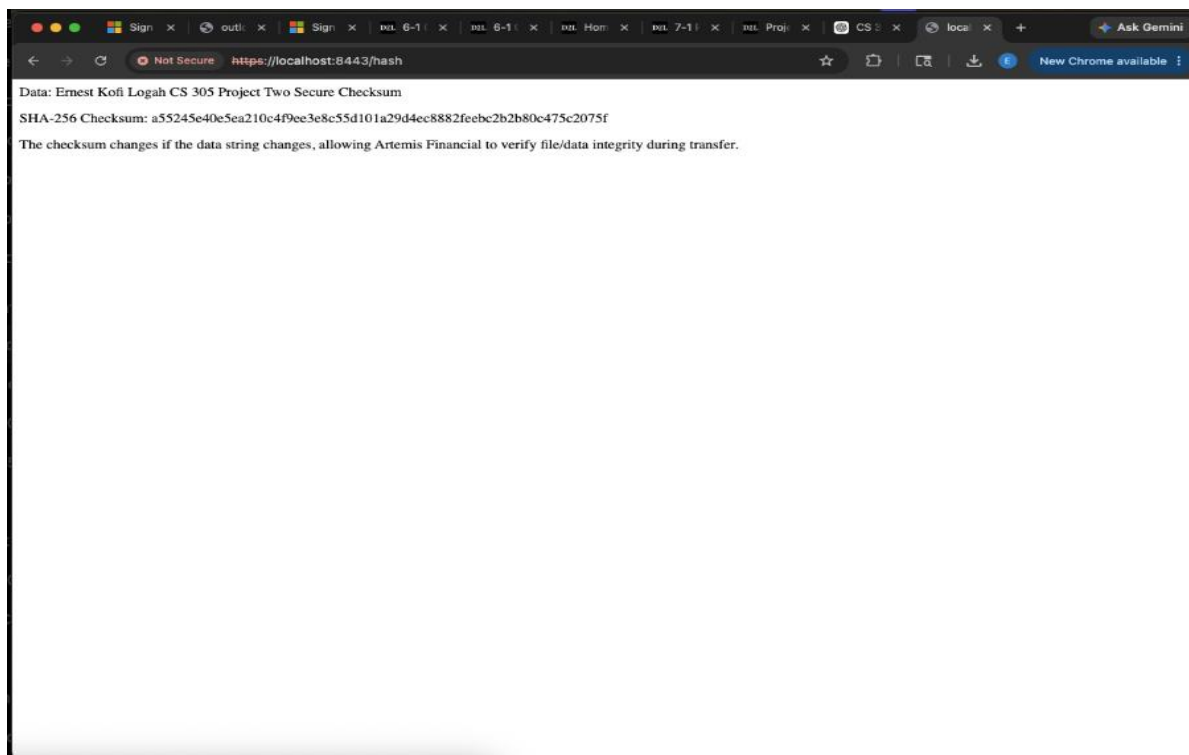


Figure 3. HTTPS communication successfully configured and operating on secure port 8443.

5. Secondary Testing

OWASP Dependency-Check version 10.0.4 was executed after refactoring to identify known vulnerabilities in third-party dependencies. The scan analyzed 49 dependencies and identified vulnerabilities within several legacy libraries, including Hibernate Validator, Jackson Databind, JSON Smart, Log4j API, and Logback Core. These findings primarily originated from dependencies already present in the original Artemis Financial code base. The security

enhancements added during this project, including SHA-256 checksum verification, HTTPS configuration, and SSL certificate deployment, did not introduce any new vulnerabilities. Performing dependency analysis demonstrates the importance of continuously monitoring third-party software components and updating vulnerable libraries whenever possible.

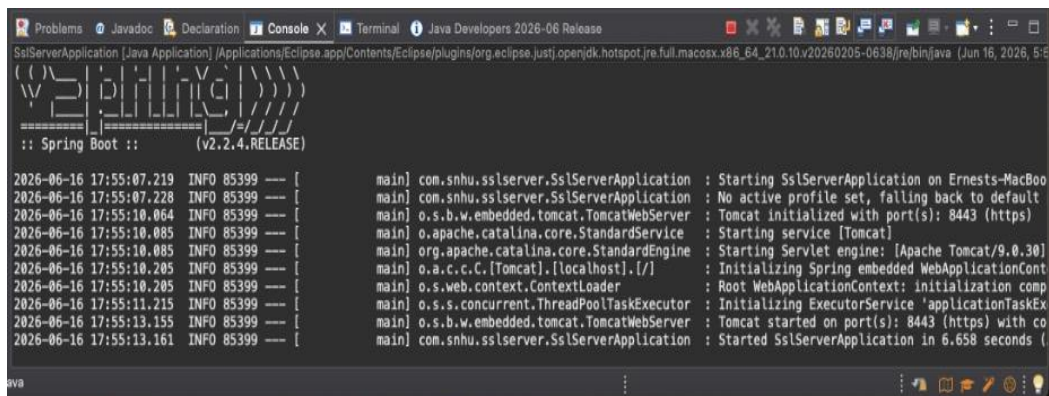
The screenshot shows the OWASP Dependency-Check web application interface. At the top, there's a navigation bar with links like 'How to read the report', 'Suppressing false positives', and 'Getting Help: github issues'. Below this, the project name 'ssl-server' and version '0.0.1-SNAPSHOT' are displayed. A 'Scan Information' section provides details about the scan, including the version (10.0.4), report generation time (Tue, 16 Jun 2026 18:59:24 -0400), and the number of dependencies scanned (49). A 'Summary' section follows, displaying a table of vulnerable dependencies.

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	HIGH	4
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	6
json-path-2.4.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*:*:*	pkg:maven/com.jayway.jsonpath/json-path@2.4.0	MEDIUM	1
json-smart-2.3.jar	cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:*:*:*	pkg:maven/net.minidev/json-smart@2.3	HIGH	2
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	HIGH	5
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2

Figure 4. OWASP Dependency-Check analysis performed after refactoring to evaluate dependency vulnerabilities.

6. Functional Testing

Functional testing verified that the application executed successfully after refactoring. Testing confirmed that the SHA-256 checksum endpoint generated the expected output, the SSL certificate loaded correctly, and HTTPS communication was established on port 8443. The application started successfully and processed requests without runtime errors. Manual review also confirmed that the checksum changes whenever the original data changes, validating the integrity verification process. Manual code review was also performed to identify potential syntactical, logical, and security-related vulnerabilities. The review confirmed that the checksum functionality correctly generated a unique SHA-256 hash, HTTPS communication was properly configured through the application properties file, and the SSL certificate was successfully loaded from the PKCS12 keystore. No runtime errors were observed during application execution, and all implemented security controls functioned as expected.



```
SslServerApplication [Java Application] /Applications/Eclipse.app/Contents/Eclipse/plugins/org.eclipse.justi.openjdk.hotspot.jre.full.macosx.x86_64_21.0.10.v20260205-0638/jre/bin/java (Jun 16, 2026, 5:55:07 PM)

:: Spring Boot :: (v2.2.4.RELEASE)

2026-06-16 17:55:07.219 INFO 85399 --- [main] com.snhu.sslserver.SslServerApplication : Starting SslServerApplication on Ernest's-MacBoo
2026-06-16 17:55:07.228 INFO 85399 --- [main] com.snhu.sslserver.SslServerApplication : No active profile set, falling back to default
2026-06-16 17:55:10.064 INFO 85399 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port(s): 8443 (https)
2026-06-16 17:55:10.085 INFO 85399 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-06-16 17:55:10.085 INFO 85399 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-06-16 17:55:10.205 INFO 85399 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-06-16 17:55:10.205 INFO 85399 --- [main] o.s.web.context.ContextLoader : Root WebApplicationContext: initialization complete
2026-06-16 17:55:11.215 INFO 85399 --- [main] o.s.s.concurrent.ThreadPoolTaskExecutor : Initializing ExecutorService 'applicationTaskEx
2026-06-16 17:55:13.155 INFO 85399 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8443 (https) with co
2026-06-16 17:55:13.161 INFO 85399 --- [main] com.snhu.sslserver.SslServerApplication : Started SslServerApplication in 6.658 seconds (
```

Figure 5. Successful application execution following security refactoring and testing.

7. Summary

The Artemis Financial application was successfully refactored to improve security and comply with software security testing protocols. Security improvements included implementing SHA-256 hashing for data integrity verification, generating a self-signed SSL certificate, enabling HTTPS communication through TLS encryption, and performing vulnerability analysis using OWASP Dependency-Check. The vulnerability assessment process included evaluating cryptographic controls, client-server communications, dependency vulnerabilities, and application execution behavior. Refactoring efforts focused on improving confidentiality through encrypted communications, integrity through checksum verification, and trust through certificate-based authentication. These enhancements strengthen the overall security posture of the application while reducing exposure to common cybersecurity risks.

8. Industry Standard Best Practices

Several industry-standard best practices were applied during this project. SHA-256 hashing was implemented to verify data integrity and detect unauthorized modifications to transmitted information. SSL/TLS encryption was configured to protect sensitive information while traveling between the client and server. Certificate-based authentication was used to establish trust and support encrypted communications. OWASP Dependency-Check was executed to identify known vulnerabilities within third-party libraries and support proactive vulnerability management. Additionally, secure coding principles were followed throughout the refactoring process to minimize security weaknesses and improve application reliability. Applying these best practices helps Artemis Financial protect customer information, reduce cybersecurity risks, support regulatory compliance efforts, and maintain customer confidence in the security of its systems.